Day 10 – Final Integration & Deployment

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Docker Running**: Ensure Docker and Docker Compose are installed and running locally.
   - **Git Configured**: Set up an active Git repository pushed to a private or public GitHub account.
   - **Render Account**: Open a browser page to `https://render.com` (or your preferred cloud provider).
   - **IDE Ready**: Open `Dockerfile`, `docker-compose.yml`, and `gamekey_platform/settings.py` in your IDE.

2. **Prerequisites**:
   - You must have completed Day 9 successfully (All tests passing locally, coverage ≥ 70%).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain containerization principles and how Docker solves environment dependency issues.
- Build a Python base image using `Dockerfile` instructions and the `uv` tool.
- Assemble multi-container architectures using `docker-compose.yml`.
- Configure binding and ports to expose applications to local hosts.
- Configure and deploy applications to Render (Web Services, Workers, and Redis).
- Complete a final system integration test checking endpoints, auth, tasks, audit logs, and tests.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	9 min	Introduce containerization. Discuss local vs. production differences.
2. Key Concepts & Core Ideas	7 min	Explain Docker images vs. containers, port bindings, compose networks, and production web servers.
3. Live Coding Walkthrough	20 min	Write the Dockerfile, write docker-compose.yml, test multi-service stack locally, and deploy services to Render.

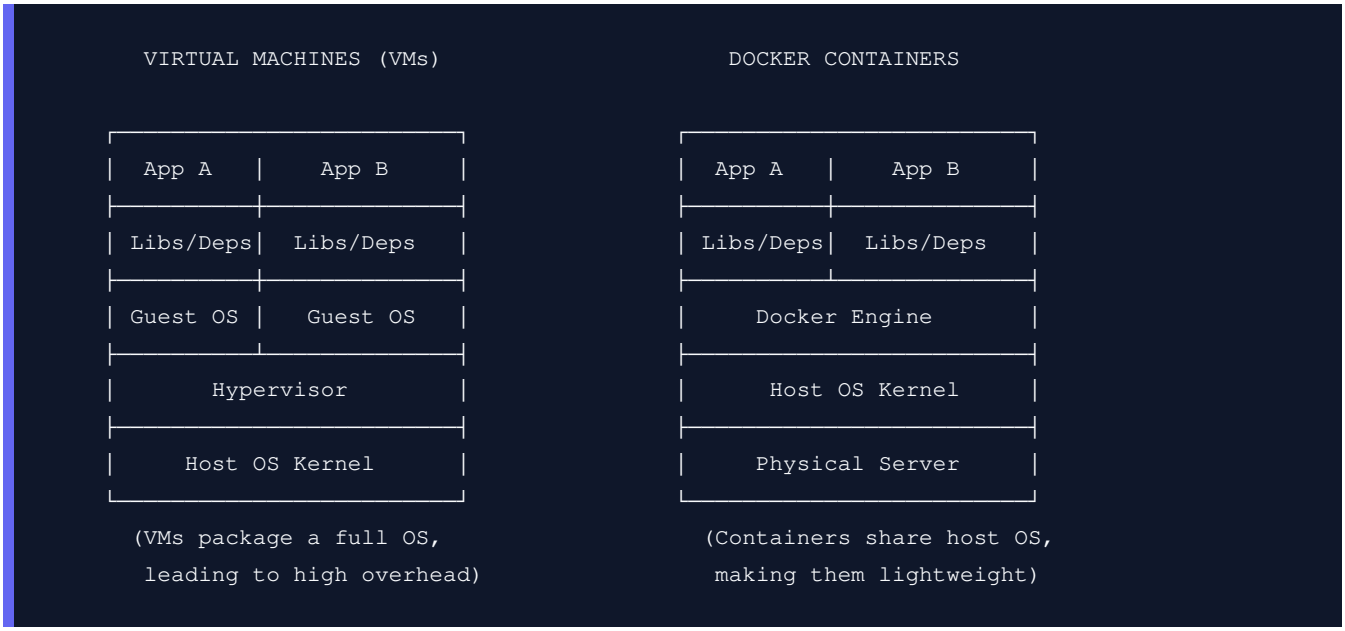
4. Check for Understanding	4 min	Q&A on build vs. runtime commands, local loops in containers, and database ephemeral disks.
5. Class Wrap-up & Wrap-up Checklist	5 min	Complete final system integration checklist. Celebrate bootcamp completion!

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (9 min)

How many times have you heard 'but it works on my machine!' when deploying code? Today, we will package our application code, virtual environment, and system packages into an immutable container image. This image will run identically on your laptop, a colleague's machine, or a cloud server in Oregon.

- **Multi-Service Architecture**: Review the final architecture. To run this app, we need three pieces: a web API server, a Redis queue, and a Celery worker. Running these individually on a server is complex; Docker Compose simplifies this by orchestrating them in a single command.
- **Q&A**:
  - **Question**: Why can't we use SQLite in production on Render?
    - *Answer*: SQLite stores all data in a single local file (`db.sqlite3`). Since Render containers have an ephemeral filesystem, any local file written to the container's disk is destroyed whenever the container is redeployed, restarted, or scaled. To keep data safe, we must use a persistent external database like PostgreSQL.



2. Key Concepts & Core Ideas (7 min)

Introduce these container and deployment rules:

- **Docker Image vs. Container**: An *image* is the read-only blueprint (like a class definition). A *container* is a running, writable instance of that image (like an instantiated object).
- **`RUN` vs. `CMD`**:
  - `RUN` executes during the image build phase (installing tools, running migrations, compiling files).
  - `CMD` defines the default shell script execution command when the container starts up.
- **`0.0.0.0` Binding**: Django's dev server binds to `127.0.0.1` by default. Inside a container, `127.0.0.1` is completely isolated. Binding to `0.0.0.0` tells the container to listen for network traffic coming from outside its boundaries (allowing the host OS to route traffic to it).
- **Service Networking**: In Docker Compose, containers communicate using service names (e.g. `redis://redis:6379/0`) rather than `localhost`.
- **Gunicorn**: Django's `runserver` is single-threaded and unsafe for production. In production, we swap `runserver` for a WSGI HTTP server like `Gunicorn` which uses multi-process worker pools.
- **Ephemeral Filesystem**: Cloud containers are stateless; any file written to local disk (like `db.sqlite3`) is destroyed on redeploy. Thus, real databases (PostgreSQL) must run outside the web container.

---

3. Live Coding Walkthrough (20 min)

Step 3.1: Writing the Dockerfile

- **Concept**: Create the image blueprint. We start from a slim Python 3.12 image, copy `uv` binary to install dependencies rapidly, copy our code, sync libraries, and set the default launch command.
- **Code**: Create `Dockerfile` in the project root:

```
# Use lightweight slim python base image to minimize final image footprint
FROM python:3.12-slim

# Copy the uv binary tool directly from its official container image
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv

# Set the active working directory inside the container's virtual filesystem
WORKDIR /app

# Copy all local project files into the container's /app directory
COPY . .

# Sync dependencies. --no-dev excludes testing tools like pytest to optimize size
RUN uv sync --no-dev

# Define startup command. Binding to 0.0.0.0 allows host routing
CMD ["uv", "run", "python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Step 3.2: Assembling the Multi-Container Compose Stack

- **Concept**: Create the compose file to wire up our web, worker, and Redis services. Notice that both and `worker` build from the same Dockerfile, but execute different commands on startup.
- **Code**: Create `docker-compose.yml` in the project root:

```
version: '3.9'

services:
  web:
    # Build the image using the local Dockerfile
    build: .
    # Bind container port 8000 to host port 8000
    ports:
      - "8000:8000"
    # Load environment variables from the local .env file
    env_file:
      - .env
    # Ensure redis container starts before web container boots up
    depends_on:
      - redis
    # Override the default Dockerfile CMD to run web server
    command: uv run python manage.py runserver 0.0.0.0:8000

  worker:
    build: .
    env_file:
      - .env
    depends_on:
      - redis
    # Override CMD to start Celery background worker
    command: uv run celery -A gamekey_platform worker --loglevel=info

  redis:
    # Fetch pre-built lightweight Redis image from Docker Hub
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

- **Verify**: Ensure YAML indentation is correct.

Step 3.3: Running the Stack Locally

- **Concept**: Compile the images and bring the services up.
- **Code**:

```
# Compile Dockerfiles and bring all three containers up in the terminal
docker compose up --build
```

- **Verify**: Watch the terminal output. Confirm that `web`, `worker`, and `redis` boot up without errors. Send a POST request to `/api/orders/` and verify that tasks process in the containerized worker terminal. ⚠ **Common Pitfalls**:

- **misconfiguration**: Ensure that inside Docker Compose, points to `redis://redis:6379/0` (the name of the container service), not `localhost`.

Step 3.4: Deploying to Render

- **Concept**: Walk through pushing the repository to GitHub and deploying to Render.
- **Steps**:
  1. Push code to a GitHub repo.
  2. Create a new **Web Service** on Render, pointing to your repo.
     - Build Command: `uv sync --no-dev && python manage.py migrate`
     - Start Command: `gunicorn gamekey_platform.wsgi:application --bind 0.0.0.0:$PORT`
  3. Create a **Redis** instance on Render and copy its connection URL.
  4. Inject Environment Variables into the Web Service:
     - `CELERY_BROKER_URL` (pasted Redis URL)
     - `SECRET_KEY` (secret token)
     - `DEBUG` (`False`)
     - `ALLOWED_HOSTS` (`['your-app.onrender.com']`)
  5. Create a **Background Worker** service pointing to the same repo:
     - Start Command: `celery -A gamekey_platform worker --loglevel=info`
     - Inject same environment variables (including `CELERY_BROKER_URL`).

---

4. Check for Understanding (4 min)

> ***Question***: *"Why do the `web` and `worker` services use the same Docker image but different `command` values?"*
> ***Answer***: *Both services run the same codebase and need the same environment variables, libraries, and settings. However, the `web` service runs the HTTP web server (`manage.py runserver` or Gunicorn) to handle API requests, while the `worker` service runs the Celery daemon (`celery worker`) to process background tasks. Using the same image simplifies building and keeps the dependencies identical.*

> ***Question***: *"What's the problem with binding the Django dev server to `127.0.0.1` inside a Docker container?"*
> ***Answer***: *`127.0.0.1` is the loopback interface, which is only accessible within the container itself. If the dev server is bound to `127.0.0.1`, the host machine and outer network cannot access the container's port. Binding to `0.0.0.0` tells the container to listen on all interfaces, allowing traffic from the host machine to reach the server.*

> ***Question***: *"Why can't we use SQLite in production on Render?"*
> ***Answer***: *SQLite stores data in a local file on the container's disk. Render containers have an ephemeral filesystem, meaning their disks are wiped and recreated on every deployment, restart, or scale event, resulting in complete database loss. Production environments require a persistent, remote database service (like PostgreSQL).*

---

5. Final Integration Checklist (5 min)

Final checklist before wrapping up:

- [ ] **API Endpoints**: All endpoints return standard HTTP status codes (`200`, `201`, `400`, `401`, `403`, `404`).
- [ ] **Security**: Token Auth blocks unauthenticated writes and custom permissions protect publisher resources.
- [ ] **Async Engine**: The management command `check_expired_keys` successfully detects expired keys and dispatches background tasks.
- [ ] **Audit Trail**: Every webhook delivery attempt is logged in `WebhookDeliveryLog` showing response statuses and retries.
- [ ] **Tests**: The unit test suite passes successfully with code coverage ≥70%.
- [ ] **Containers**: Docker Compose spins up the web API, message queue, and Celery worker cleanly.

📂 Project Structure (End of Day 10)

```
gamekey_platform/
├── gamekey_platform/
│   ├── __init__.py          # Loads Celery app
│   ├── celery.py
│   ├── settings.py
│   └── urls.py
├── games/
│   ├── management/
│   │   └── commands/
│   │       └── check_expired_keys.py
│   ├── migrations/
│   ├── tests/
│   │   └── test_webhook.py
│   ├── admin.py
│   ├── models.py            # Publisher, Game, GameKey, Order, OrderItem, WebhookDeliveryLog
│   ├── permissions.py
│   ├── serializers.py
│   ├── tasks.py             # Celery async webhook task
│   ├── viewsets.py
│   ├── views.py             # register, create_order
```

```
|   └── webhooks.py          # sync helper (Day 7 reference)
├── .env
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── manage.py
├── pytest.ini
└── README.md
```

🔌 API Reference — Sample Requests & Responses

All endpoints are prefixed with `/api/`. Authenticated endpoints require the header:

```
Authorization: Token <your-token>
```

POST `/api/register/`

Register a new user and receive an auth token.

- **Request**

```
POST /api/register/
Content-Type: application/json

{
  "username": "dheeresh",
  "password": "securepass123"
}
```

- **Response `201 Created`**

```
{
  "token": "9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a"
}
```

- **Response `400 Bad Request`** — missing fields

```
{
  "error": "Username and password required."
}
```

GET `/api/publishers/`

List all publishers. Public endpoint (no auth needed).

- **Request**

```
GET /api/publishers/
```

- **Response `200 OK`**

```
[
  {
    "id": 1,
    "name": "Valve Corporation",
    "webhook_url": "https://valve.example.com/webhooks/gamekey",
    "user": 2
  },
  {
    "id": 2,
    "name": "CD Projekt Red",
    "webhook_url": "https://cdpr.example.com/hooks/expiry",
    "user": 3
  }
]
```

(Note: `webhook_secret` is always excluded from responses)

POST `/api/publishers/`

Create a publisher profile. Requires auth.

- **Request**

```
POST /api/publishers/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "webhook_secret": "mysecret_abc123",
  "user": 2
}
```

- **Response `201 Created`**

```
{
  "id": 1,
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "user": 2
}
```

GET `/api/games/`

List all games. Public endpoint.

- **Request**

```
GET /api/games/
```

- **Response `200 OK`**

```
[
  {
    "id": 1,
    "title": "Half-Life 3",
    "publisher": 1,
    "price": "29.99"
  },
  {
    "id": 2,
    "title": "Cyberpunk 2077",
    "publisher": 2,
    "price": "49.99"
  }
]
```

POST `/api/games/`

Create a game. Requires auth.

- **Request**

```
POST /api/games/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
```

```
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

- **Response** `201 Created`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

GET `/api/games/{id}/`

Retrieve a single game.

- **Request**

```
GET /api/games/1/
```

- **Response** `200 OK`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

- **Response** `404 Not Found`

```
{
  "detail": "No Game matches the given query."
}
```

PATCH `/api/games/{id}/`

Partially update a game. Requires auth + must be the publisher's owner.

- **Request**

```
PATCH /api/games/1/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "price": "19.99"
}
```

- **Response** `200 OK`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "19.99"
}
```

- **Response** `403 Forbidden`

```
{
  "detail": "You do not have permission to perform this action."
}
```

POST `/api/orders/`

Buy a game key. Requires auth. Atomically assigns or generates a key with a 30-day expiry.

- **Request**

```
POST /api/orders/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "game_id": 1
}
```

- **Response** `201 Created`

```
{
  "order_id": 42,
  "game": "Half-Life 3",
  "key": "A3F9-B21C-4DE7-9901-CC84",
  "expires_at": "2026-07-16T10:30:00Z"
```

```
}
```

Webhook Payload — `game_key.expired`

Sent asynchronously by Celery to the publisher's `webhook_url` when a key expires. Signed with HMAC-SHA256.

- **Headers**

```
Content-Type: application/json
X-Signature: sha256=3c6e9b52a3c2ac3f7dca0c944b6d68b3f15ea2e3c0cf1b2e5f7a9d4c8e1f230b
```

- **Body**

```
{
  "event": "game_key.expired",
  "game_key": "A3F9-B21C-4DE7-9901-CC84",
  "game_title": "Half-Life 3",
  "expired_at": "2026-07-16T10:30:00Z",
  "attempt": 0
}
```

(Note: `attempt` increments on each retry, max 3. Exponential backoff delays: 0 → 1: 60s, 1 → 2: 120s, 2 → 3: 240s).